РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

дисциплина: Основы администрирования операционных систем

Студент: Лемуш Мариу Франсишку

Студ. билет № 1032239162

Группа: НПИбд-01-24

МОСКВА

2025 г.

Цель работы:

Целью данной работы является получение навыков управления процессами операционной системы.

Выполнение работы:

Управление заданиями:

Для начала получим полномочия администратора **su –** и введём следующие команды:

sleep 3600 &

dd if=/dev/zero of=/dev/null &

sleep 7200

Поскольку мы запустили последнюю команду без & после неё, у нас есть 2 часа, прежде чем мы снова получим контроль над оболочкой.

Введём **Ctrl + z** , чтобы остановить процесс. Затем введём **jobs** и увидим три задания, которые мы только что запустили. Первые два имеют состояние Running, а последнее задание в настоящее время находится в состоянии Stopped. Для продолжения выполнения задания 3 в фоновом режиме введём **bg 3** и с помощью команды jobs посмотрим изменения в статусе заданий (Рис. 1.1):

```
root@vbox:~# sleep 3600 &
[1] 5942
root@vbox:~# dd if=/dev/zero of=/dev/null &
[2] 5954
root@vbox:~# sleep 7200
^Z
[3]+  Stopped                 sleep 7200
^Z[1]   Concluído               sleep 3600

[3]+  Parado                  sleep 7200
root@vbox:~# jobs
[2]-  Executando              dd if=/dev/zero of=/dev/null &
[3]+  Parado                  sleep 7200
root@vbox:~# bg 3
[3]+ sleep 7200 &
root@vbox:~# jobs
```

Рис. 1.1. Получение полномочий администратора, ввод трёх команд, остановка процесса, установка выполнения задания 3 в фоновом режиме, просмотр изменений в статусе заданий.

Для перемещения задания 1 на передний план введём **fg 1**, далее введём **Ctrl + c**, чтобы отменить задание 1. С помощью команды jobs посмотрим изменения в статусе заданий и проделаем то же самое для отмены заданий 2 и 3 (Рис. 1.2):

```
root@vbox:~# jobs
[2]-  Executando              dd if=/dev/zero of=/dev/null &
[3]+  Executando              sleep 7200 &
root@vbox:~# fg 2
dd if=/dev/zero of=/dev/null
fg 1
^C84975786+0 records in^Z
[2]+  Parado                  dd if=/dev/zero of=/dev/null
root@vbox:~# jobs
[2]+  Parado                  dd if=/dev/zero of=/dev/null
[3]-  Executando              sleep 7200 &
root@vbox:~# fg 3
sleep 7200
```

Рис. 1.2. Перемещение заданий на передний план и их последующая отмена.

Теперь откроем второй терминал и под учётной записью пользователя введём в нём: **dd if=/dev/zero of=/dev/null &**. После введём **exit**, чтобы закрыть второй терминал (Рис. 1.3):

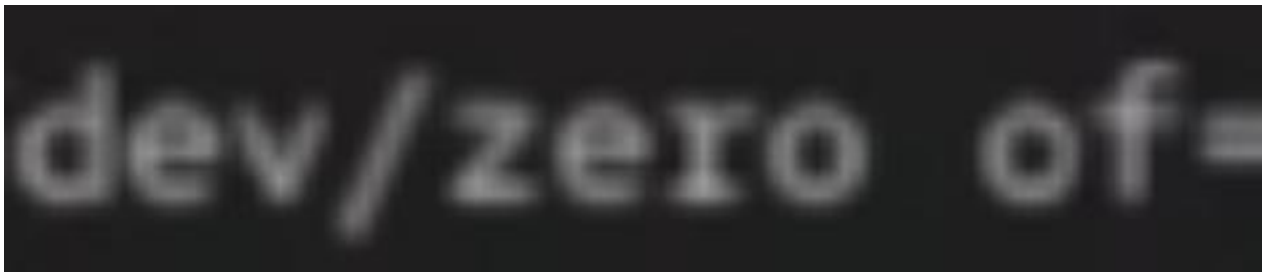

Рис. 1.3. Ввод команды и закрытие терминала.

На другом терминале под учётной записью своего пользователя запустим **top.** Мы увидим, что задание dd всё ещё запущено. Для выхода из top используем **q** и вновь запусткаем top, в нём используем **k**, чтобы убить задание dd. После этого выйдем из top (Рис. 1.4):

```
 up  4:52,  5 users,  load average: 1,13, 0,88, 0,9
al,   2 running, 256 sleeping,   0 stopped,   0 zom
us, 16,5 sy,  0,0 ni, 51,7 id,  0,0 wa,  0,0 hi,  0
53,2 total,   1339,1 free,   1865,5 used,    686,0
40,0 total,   4040,0 free,      0,0 used.   1787,7

    PR  NI    VIRT    RES    SHR S  %CPU  %MEM
    20   0  227164   2184   2064 R  90,8   0,1   2:
    20   0 4719384 388312 133004 S  40,7  10,4   4:
    20   0 2547024 409864 101060 S   4,6  11,0   0:
    20   0       0      0      0 I   0,7   0,0   0:
    20   0       0      0      0 I   0,7   0,0   0:
    20   0       0      0      0 I   0,7   0,0   0:
    20   0  231584   5412   3268 R   0,7   0,1   0:
    20   0       0      0      0 I   0,3   0,0   0:
    20   0  530836   7740   6452 S   0,3   0,2   0:
    20   0  244036   8088   6692 S   0,3   0,2   0:
    20   0       0      0      0 I   0,3   0,0   0:
    20   0   50184  42040  10552 S   0,0   1,1   0:
    20   0       0      0      0 S   0,0   0,0   0:
    20   0       0      0      0 S   0,0   0,0   0:
     0 -20       0      0      0 I   0,0   0,0   0:
```

Рис. 1.4. Убийство задания dd в top.

Управление процессами:

Получим полномочия администратора **su -** и введём следующие команды:

dd if=/dev/zero of=/dev/null &

dd if=/dev/zero of=/dev/null &

dd if=/dev/zero of=/dev/null &

После чего введём **ps aux | grep dd**, которое показывает все строки, в которых есть буквы dd. Запущенные процессы dd идут последними. Используем PID первого процесса dd, чтобы изменить приоритет (**renice -n 5 2682**) (Рис. 2.1).

```
 login: sex jan 30 03:03:15 MSK 2026 em pts/1
box:~# dd if=/dev/zero of=/dev/null &
61
box:~# dd if=/dev/zero of=/dev/null &
64
box:~# dd if=/dev/zero of=/dev/null &
67
box:~# ps aux | grep dd
         2  0.0  0.0      0     0 ?        S    00:14   0:00 [kthr
        12  0.0  0.0      0     0 ?        I    00:14   0:00 [kwor
v6_addrconf]
        85  0.0  0.0      0     0 ?        I<   00:14   0:00 [kwor
ddrconf]
      3853  0.0  0.7 964392 26800 ?        Ssl  00:16   0:00 /usr/
ution-addressbook-factory
      4595  0.0  0.2 609776  9704 ?        Sl   00:17   0:01 /usr/
d-wsdd --spawner :1.17 /org/gtk/gvfs/exec_spaw/4
      4600  0.0  0.7 260152 29764 ?        S    00:17   0:02 /usr/
/usr/bin/wsdd --no-host --discovery --listen /run/user/1000/gvfsd/
      6265 91.4  0.0 227164  2184 pts/1    R    05:04   5:37 dd if
f=/dev/null
      6461 65.8  0.0 227164  2168 pts/1    R    05:10   0:28 dd if
```

Рис. 2.1. Получение полномочий администратора, ввод команд. Просмотр всех строк, в которых есть dd. Изменение приоритета.

Введём **ps fax | grep -B5 dd.** Параметр -B5 показывает соответствующие запросу строки, включая пять строк до этого. Поскольку ps fax показывает иерархию отношений между процессами, мы также видим оболочку, из которой были запущены все процессы dd, и её PID (Рис. 2.2).

```
251  0.0  0.0      0     0 ?        I    05:03   0:0
-1]
255  0.0  0.0      0     0 ?        I    05:04   0:0
253:0-1]
265 90.2  0.0 227164  2184 pts/1    R    05:04   7:5
/null

382  0.0  0.0      0     0 ?        I    05:08   0:0

388  0.0  0.1 232628  6128 pts/1    S    05:08   0:0
403  0.0  0.1 230108  5620 pts/1    S    05:08   0:0
458  0.0  0.1  16308  6136 ?        S    05:10   0:0
ng...
459  0.0  0.1  16308  6088 ?        S    05:10   0:0
ng...
461 48.7  0.0 227164  2168 pts/1    RN   05:10   1:3
/null
464 62.2  0.0 227164  2244 pts/1    R    05:10   2:0
/null
467 61.1  0.0 227164  2184 pts/1    R    05:10   1:5
/null
468  0.0  0.1  16308  6132 ?        S    05:10   0:0
```

Рис. 2.2. Просмотр иерархии отношений между процессами.

Теперь найдём PID корневой оболочки, из которой были запущены процессы dd, и введём kill -9 (указав PID оболочки). Мы увидим, что наша корневая оболочка закрылась, а вместе с ней и все процессы dd (остановка родительского процесса — простой и удобный способ остановить все его дочерние процессы) (Рис. 2.3).

```
root@vbox:~# kill -9 2654
```

Рис. 2.3. Закрытие корневой оболочки.

Самостоятельная работа (задание 1):

Получим полномочия администратора **su** – и запустим команду **dd if=/dev/zero of=/dev/null &** трижды как фоновое задание. Затем увеличим приоритет первой команды, используя значение приоритета −5, после чего изменим приоритет того же процесса ещё раз, но используем на этот раз значение −15 (мы можем менять приоритет команды от -20 (самый высокий приоритет) до 19 (самый низкий приоритет)). Завершим все процессы dd, которые мы запустили командой: **killall dd** (Рис. 3).

```
[4] 6553
root@vbox:~# dd if=/dev/zero of=/dev/null &
[5] 6556
root@vbox:~# dd if=/dev/zero of=/dev/null &
[6] 6559
root@vbox:~# renice -n 5 6553
6553 (process ID) old priority 0, new priority 5
root@vbox:~# renice -n 15 6553
6553 (process ID) old priority 5, new priority 15
root@vbox:~# killall dd
[4]   Terminado              dd if=/dev/zero of=/dev/null
```

Рис. 3. Получение полномочий администратора, запуск команды трижды как фоновое задание, увеличение приоритета первой команды, завершение всех процессов.

Самостоятельная работа (задание 2):

Получим полномочия администратора **su –** и запустим программу yes в фоновом режиме с подавлением потока вывода (**yes > /dev/null &**), далее запустим программу yes на переднем плане с подавлением потока вывода и приостановим выполнение программы. Заново запустим программу yes с теми же параметрами, затем завершим её выполнение. Повторим действия, но уже запустим программу yes на переднем плане без подавления потока вывода (**yes > /dev/null**). Также приостановим выполнение программы и заново запустим программу yes с теми же параметрами, затем завершим её выполнение. Проверим состояния заданий, воспользовавшись командой **jobs**. Далее переведём процесс, который у нас выполняется в фоновом режиме, на передний план, затем остановим его (**fg 1, после чего Ctrl+c**). Переведём 3 процесс с подавлением потока вывода в фоновый режим (**bg 3**) и проверим состояния заданий, воспользовавшись командой **jobs**. Обратим внимание, что процесс стал выполняющимся (Running) в фоновом режиме. Запустим процесс в фоновом режиме таким образом, чтобы он продолжил свою работу даже после отключения от терминала (**nohup yes > /dev/null &**). Закроем окно и заново запустим консоль. Убедимся, что процесс продолжил свою работу (Рис. 4.1).

```
^Z
[5] 6626
-bash: :s^Z: substitution failed
root@vbox:~# yes > /dev/null &
^Z
[6] 6650
-bash: :s^Z: substitution failed
root@vbox:~# jobs
[4]   Executando              yes > /dev/null &
[5]-  Executando              yes > /dev/null &
[6]+  Executando              yes > /dev/null &
root@vbox:~# fg 1
-bash: fg: 1: trabalho não existe
root@vbox:~# yes > /dev/null &
[7] 6662
root@vbox:~# bg 3
-bash: bg: 3: trabalho não existe
root@vbox:~# jobs
[4]   Executando              yes > /dev/null &
```

Рис. 4.1. Получение полномочий администратора. Запуск программы yes в фоновом режиме с подавлением потока вывода. Запуск программы yes на переднем плане без подавления потока вывода. Перевод процесса на передний план и его остановка. Перевод процесса в фоновый режим. Проверка состояния заданий. Запуск процесса в фоновом режиме с условиями.

Сейчас получим информацию о запущенных в операционной системе процессах с помощью утилиты **top** (Рис. 4.2).

```
35 up  5:15,  5 users,  load average: 6,51, 4,43, 3,40
otal,   6 running, 254 sleeping,   0 stopped,   0 zomb
6 us, 45,1 sy,  0,0 ni,  0,1 id,  0,0 wa,  0,0 hi,  1,
3653,2 total,   1280,5 free,   1921,1 used,    689,8 b
4040,0 total,   4040,0 free,      0,0 used.   1732,1 a

      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     T
      20   0  227136   2112   1988 R  51,0   0,1   2:1
      20   0  227136   2160   2036 R  43,6   0,1   0:4
      20   0  227136   2240   2120 R  41,7   0,1   3:1
      20   0  227136   2208   2088 R  39,7   0,1   2:4
      20   0  227136   2216   2096 R  39,1   0,1   1:3
os    20   0 4721508 390428 133284 S  29,5  10,4   6:4
os    20   0 2556936 432324 104796 S  10,6  11,6   2:2
os    20   0 2485156 402520 111228 S   6,7  10,8   0:1
      20   0       0      0      0 I   5,8   0,0   0:0
os     9 -11  323884  13000   7908 S   2,6   0,3   0:0
      20   0   16308   6140   5136 S   1,6   0,2   0:0
      20   0  231596   5384   3220 R   1,3   0,1   0:0
      20   0       0      0      0 I   0,6   0,0   0:0
      20   0   50184  42040  10552 S   0,3   1,1   0:0
      20   0       0      0      0 I   0,3   0,0   0:1
      20   0   36596  12348   8212 S   0,3   0,3   0:0
```

Рис. 4.2. Получение информации о запущенных в операционной системе процессах.

Запустим ещё три программы yes в фоновом режиме с подавлением потока вывода (**yes > /dev/null &**). Убьём два процесса: для одного используем его PID (**kill -9 3098**), а для другого — его идентификатор конкретного задания (**fg 2 и Ctrl+c**). Попробуем послать сигнал 1 (SIGHUP) процессу, запущенному с помощью nohup (**kill -1 3100**), и обычному процессу (**kill -1 2993**) (Рис. 4.3).

```
root@vbox:~# kill -9 6814
root@vbox:~# fg 2
-bash: fg: 2: trabalho não existe
[10]   Morto                  yes > /dev/null
root@vbox:~# kill -1 3100
-bash: kill: (3100) - Processo inexistente
root@vbox:~# kill -1 6843
-bash: kill: (6843) - Processo inexistente
```

Рис. 4.3. Запуск трёх программ yes в фоновом режиме с подавлением потока вывода, убийство двух процессов, попытка послать сигнал 1 (SIGHUP).

Запустим ещё несколько программ yes в фоновом режиме с подавлением потока вывода (**yes > /dev/null &**) и завершим их работу одновременно, используя команду **killall yes** (Рис. 4.4).

```
root@vbox:~# yes > /dev/null &
[14] 6849
root@vbox:~# yes > /dev/null &
```

Рис. 4.4. Запуск программ yes в фоновом режиме с подавлением потока вывода и одновременное завершение их работы.

После чего запустим программу yes в фоновом режиме с подавлением потока вывода (**yes > /dev/null &**). Используя утилиту nice (**nice -n 15 yes**), запустим программу yes с теми же параметрами и с приоритетом, большим на 5. Сравним абсолютные и относительные приоритеты у этих двух процессов (**ps -l | grep yes**). Используя утилиту renice, изменим приоритет у одного из потоков yes таким образом, чтобы у обоих потоков приоритеты были равны (**renice -n 15 3109**) (Рис. 4.5).

```
[11]-  Fim da execução com status 127        nice -5 15 yes > /dev/null
root@vbox:~# nice:  "15": no such file or directory
bash: nice:: comando não encontrado...
O comando similar é: 'nice'
root@vbox:~# "nice":  "15": no such file or directory
bash: nice:: comando não encontrado...
O comando similar é: 'nice'
root@vbox:~# nice -5 15 yes > /dev/null &
[11] 6929
root@vbox:~# nice: "15": Arquivo ou diretório inexistente

[11]-  Fim da execução com status 127        nice -5 15 yes > /dev/null
root@vbox:~# ps -l | grep yes
0 R     0    6871    6403 91  80   0 - 56784 -      pts/1    00:05:18 yes
root@vbox:~# renice -n 15 6871
```

Рис. 4.5. Запуск программы yes в фоновом режиме с подавлением потока вывода. Запуск программы yes с теми же параметрами и с приоритетом, большим на 5. Сравнение абсолютных и относительных приоритетов, изменение приоритета.

Ответы на контрольные вопросы:

1. Какая команда даёт обзор всех текущих заданий оболочки? **jobs.**

```
root@vbox:~# bg 3
```

2. Как остановить текущее задание оболочки, чтобы продолжить его выполнение в фоновом режиме? **bg номер_задания.**

```
root@vbox:~# bg 3
```

3. Какую комбинацию клавиш можно использовать для отмены текущего задания оболочки? **Ctrl+c.**

```
^C
```

4. Необходимо отменить одно из начатых заданий. Доступ к оболочке, в которой в данный момент работает пользователь, невозможен. Что можно

сделать, чтобы отменить задание? **Внутри top использовать k, чтобы убить задание.**

```
4616 mlemos    20   0 2558332 433652 104820 S   4,9  11,6   3:21.64
3938 mlemos     9 -11  324604  13008   7908 S   2,3   0,3   0:05.82
3942 mlemos     9 -11  782836  24292  18972 S   2,3   0,6   0:04.00
3943 mlemos     9 -11  399876  18724   7160 S   1,6   0,5   0:01.58
3543 mlemos    20   0  607208  12260   6960 S   0,7   0,3   0:09.35
4067 mlemos    20   0  743424  37084  28500 S   0,7   1,0   0:02.39
5518 root      20   0       0      0      0 I   0,7   0,0   0:07.33
  18 root      20   0       0      0      0 I   0,3   0,0   0:12.23
3553 mlemos    20   0  850160  21900  18208 S   0,3   0,6   0:01.30
```

5. Какая команда используется для отображения отношений между родительскими и дочерними процессами? **ps fax.**

```
4264 ?        Ssl    0:00  \_ /usr/libexec/dconf-service
4321 ?        S      0:00  \_ catatonit -P
4616 ?        Rsl    3:25  \_ /usr/bin/ptyxis --gapplication-service
4623 ?        Ssl    0:09  |   \_ /usr/libexec/ptyxis-agent --socket-fd=3 -
4663 pts/0    Ss     0:00  |       \_ /usr/bin/bash
4693 pts/0    S      0:00  |       |   \_ su -
4729 pts/0    S+     0:00  |       |       \_ -bash
5844 pts/1    Ss     0:00  |       \_ /usr/bin/bash
6388 pts/1    S      0:00  |           \_ su -
```

6. Какая команда позволит изменить приоритет процесса с идентификатором 1234 на более высокий? **renice -n приоритет_процесса <PID>.**

```
root@vbox:~# renice -n 15 3942
3942 (process ID) old priority -11, new priority 15
```

7. В системе в настоящее время запущено 20 процессов dd. Как проще всего остановить их все сразу? **killall dd.**

```
vbox:~# killall dd
```

8. Какая команда позволяет остановить команду с именем mycommand? **Сначала узнаем PID процесса mycommand -ps aux | grep mycommand далее команда kill -9 <PID>.**

9. Какая команда используется в top, чтобы убить процесс? **k.**

6871	**root**	**35**	**15**	**227136**	**2188**	**2068**	**R**	**91,4**	**0,1**	**13:04.06**
3449	mlemos	20	0	4717348	390324	133284	S	18,8	10,4	9:06.82
4616	mlemos	20	0	2558332	433652	104820	S	13,8	11,6	3:31.02
6856	root	20	0	0	0	0	I	4,3	0,0	0:01.01
5667	root	20	0	0	0	0	I	3,6	0,0	0:05.01
6934	root	20	0	0	0	0	I	1,6	0,0	0:02.08
18	root	20	0	0	0	0	I	0,7	0,0	0:12.59
7106	**root**	**20**	**0**	**231596**	**5424**	**3264**	**R**	**0,7**	**0,1**	**0:00.08**
6931	root	20	0	0	0	0	I	0,3	0,0	0:00.24

10. Как запустить команду с достаточно высоким приоритетом, не рискуя, что не хватит ресурсов для других процессов? **Запустить команду в фоновом режиме.**

Вывод:

В ходе выполнения лабораторной работы были получены навыки управления процессами операционной системы.